SQL Server Comprehensive Learning Notes

Table of Contents

SELECT Queries - Data Retrieval

Definition

SELECT queries are used to retrieve and display data from database tables. They form the foundation of data analysis and reporting in SQL.

Keyboard Shortcut

- **Ctrl + R** - Toggle the results window in SQL Server Management Studio (SSMS)

SELECT Query Order of Execution

SQL processes SELECT queries in a specific order, not the order you write them:

```
/* Execution Order: */
SELECT (5th)
DISTINCT (2nd)
TOP (3rd)
FROM (1st)
WHERE (2nd)
GROUP BY (4th)
HAVING (6th)
ORDER BY (7th)
```

1. Basic SELECT - Retrieving All Data

Purpose: Select all columns from a table

Syntax:

```
SELECT * FROM table_name;
```

Example:

```
-- Select all customers
SELECT * FROM customers;

-- Select all orders
SELECT * FROM orders;
```

Use Case:

- Quick data exploration
- Verifying table structure
- Initial data inspection before analysis

---

2. Column Selection - Retrieving Specific Columns

Purpose: Select only the columns you need for analysis

Syntax:

```
SELECT column1, column2, column3 FROM table_name;
```

Example:

```
-- Select only country and score columns
SELECT country, score FROM customers;
```

Use Case:

- Performance optimization (retrieve only necessary data)
- Privacy (hide sensitive columns)
- Cleaner result sets focused on specific analysis

---

3. ORDER BY - Sorting Results

Purpose: Sort query results in ascending or descending order

Syntax:

```
SELECT * FROM table_name ORDER BY column_name ASC|DESC;
```

Parameters:

- **ASC** (Ascending): Default order, smallest to largest
- **DESC** (Descending): Largest to smallest

Examples:

```
-- Order by score in ascending order (low to high)
SELECT * FROM customers ORDER BY score ASC;

-- Order by score in descending order (high to low)
SELECT * FROM customers ORDER BY score DESC;

-- Multiple column sorting (nested order)
SELECT * FROM customers ORDER BY country ASC, score DESC;
-- This sorts by country first, then by score within each country
```

Use Case:

- Ranking customers by sales
- Finding top/bottom performers
- Chronological data analysis
- Multi-level sorting for complex analysis

---

4. GROUP BY - Data Aggregation

Purpose: Group rows with the same values and perform aggregate calculations on each group

Syntax:

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

Critical Rule - GROUP BY Rule:

> All columns in the SELECT clause must be either:
>
> 1. Used in an aggregate function (SUM, COUNT, AVG, MIN, MAX), OR
> 2. Included in the GROUP BY clause

Examples:

```
-- Group customers by country and find total score per country
SELECT
    country,
    SUM(score) AS total_score
FROM customers
GROUP BY country
ORDER BY country DESC;

-- Count customers and sum scores by country
SELECT
    country,
    COUNT(id) AS total_customers,
    SUM(score) AS total_score
FROM customers
GROUP BY country
ORDER BY total_score DESC;
```

Use Case:

- Sales analysis by region
- Customer segmentation by country/category
- Performance metrics by team/department
- Inventory analysis by product category

---

5. DISTINCT - Removing Duplicates

Purpose: Remove duplicate values from results, showing each unique value only once

Syntax:

```
SELECT DISTINCT column_name FROM table_name;
```

Example:

```
-- Get unique list of countries (no duplicates)
SELECT DISTINCT country FROM customers;
```

⚠ **Performance Note**:

- Don't use DISTINCT unless necessary
- It can slow down queries significantly as SQL must scan entire result set
- Avoid on large datasets with many unique values

Use Case:

- Finding unique customer countries
- Identifying all product categories
- Discovering distinct values in a column
- Data profiling and exploration

---

6. HAVING Clause - Post-Aggregation Filtering

Purpose: Filter aggregated results (filters data AFTER grouping and aggregation)

Key Differences from WHERE:

- **WHERE**: Filters rows BEFORE aggregation
- **HAVING**: Filters aggregated results AFTER GROUP BY
- **HAVING** can only be used with GROUP BY
- **HAVING** can only filter on aggregated columns or grouped columns

Syntax:

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

Examples:

```
-- Find countries with total score > 500
SELECT
    country,
    COUNT(id) AS total_customers,
    SUM(score) AS total_score
FROM customers
GROUP BY country
HAVING SUM(score) > 500
ORDER BY total_score DESC;
/* Explanation:
   - WHERE score>500 would filter individual rows BEFORE grouping
   - HAVING SUM(score)>500 filters groups AFTER aggregation
   - Only countries with combined score exceeding 500 appear in results */

-- Find countries with total score < 860
SELECT
    country,
    COUNT(id) AS total_customers,
    SUM(score) AS total_score
FROM customers
GROUP BY country
HAVING SUM(score) < 860
ORDER BY total_score DESC;
```

```sql
-- Find countries with average score > 430 (excluding zero scores)
SELECT
    country,
    AVG(score) AS avg_score
FROM customers
WHERE score != 0  /* WHERE filters before grouping */
GROUP BY country
HAVING AVG(score) > 430  /* HAVING filters after grouping */
ORDER BY avg_score DESC;
/* Explanation:
   - WHERE score!=0 removes zero scores from calculation
   - GROUP BY country groups remaining data
   - HAVING AVG(score)>430 filters groups with average > 430
   - Only countries meeting the criteria are returned */
```

Use Case:

- Find countries with minimum sales targets
- Identify departments above average performance
- Filter product categories by minimum order count
- Business intelligence filtering on KPIs

---

7. TOP - Limiting Result Rows

Purpose: Restrict the number of rows returned in results

Syntax:

```sql
SELECT TOP number_of_rows * FROM table_name;
```

Example:

```sql
-- Retrieve top 3 customers with highest scores
SELECT TOP 3 * FROM customers ORDER BY score DESC;

-- Retrieve top 2 customers with lowest scores
SELECT TOP 2 * FROM customers ORDER BY score ASC;

-- Retrieve the 3 most recent orders
SELECT TOP 3 * FROM orders ORDER BY order_date ASC;
```

Use Case:

- Leaderboards (top 10 customers by sales)
- Sample data retrieval (first 100 rows for testing)
- Pagination (first N results per page)
- Performance optimization (limit large result sets)

WHERE Operators - Data Filtering

Definition

WHERE operators filter rows based on conditions, returning only data that meets specified criteria.

Categories of WHERE Operators

---

1. Comparison Operators

Purpose: Compare values to filter rows

Operators:

- `=` Equal to
- `<>` or `!=` Not equal to (both work in SQL Server)
- `>` Greater than
- `<` Less than
- `>=` Greater than or equal to
- `<=` Less than or equal to

Examples:

```sql
-- Get customers from Germany
SELECT * FROM customers WHERE country = 'Germany';

-- Get customers NOT from Germany (both syntaxes work)
SELECT * FROM customers WHERE country != 'Germany';
SELECT * FROM customers WHERE country <> 'Germany';

-- Get customers with score > 500
SELECT * FROM customers WHERE score > 500;

-- Get customers with score >= 500
SELECT * FROM customers WHERE score >= 500;

-- Get customers with score < 500
SELECT * FROM customers WHERE score < 500;

-- Get customers with score <= 500
SELECT * FROM customers WHERE score <= 500;
```

Use Case:

- Filter by numeric ranges
- String exact matching
- Numeric comparisons for thresholds

---

2. Logical Operators

Purpose: Combine multiple conditions in WHERE clause

AND Operator

Purpose: All conditions must be TRUE to return rows (AND = strict requirement)

Syntax:

```
SELECT * FROM table_name
WHERE condition1 AND condition2 AND condition3;
```

Example:

```
-- Get customers from USA with score > 500 (BOTH conditions required)
SELECT * FROM customers
WHERE country = 'USA' AND score > 500;
```

OR Operator

Purpose: At least ONE condition must be TRUE to return rows (OR = flexible)

Syntax:

```
SELECT * FROM table_name
WHERE condition1 OR condition2 OR condition3;
```

Example:

```
-- Get customers from USA OR score > 500 (EITHER condition suffices)
SELECT * FROM customers
WHERE country = 'USA' OR score > 500;
```

NOT Operator

Purpose: Reverse the condition - returns rows that DON'T meet the criteria

Syntax:

```
SELECT * FROM table_name WHERE NOT condition;
```

Example:

```sql
-- Get customers where score is NOT > 500 (same as score <= 500)
SELECT * FROM customers WHERE NOT score > 500;

-- Get customers where score is NOT > 500 AND country is NOT USA
SELECT * FROM customers
WHERE NOT (score > 500) AND NOT (country = 'USA');
```

Use Case:

- Exclusion filtering
- Finding missing data
- Reverse logic conditions

---

3. Range Operators - BETWEEN

Purpose: Find values within a specified range (inclusive on both ends)

Syntax:

```sql
SELECT * FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

Note: Values specified are **INCLUSIVE** (include the boundary values)

Example:

```sql
-- Get customers with score between 100 and 500 (includes 100 and 500)
SELECT * FROM customers WHERE score BETWEEN 100 AND 500;

-- Equivalent to:
SELECT * FROM customers WHERE score >= 100 AND score <= 500;

-- Get orders placed between specific dates
SELECT * FROM orders WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31';
```

Use Case:

- Age range filtering
- Price range searches
- Date range queries
- Performance level bracketing

---

4. Membership Operators - IN / NOT IN

Purpose: Check if a value exists in a list of values (cleaner alternative to multiple OR conditions)

Syntax:

```
SELECT * FROM table_name WHERE column_name IN (value1, value2, value3);
SELECT * FROM table_name WHERE column_name NOT IN (value1, value2, value3);
```

Examples:

```
-- Get customers from Germany or USA using IN
SELECT * FROM customers WHERE country IN ('Germany', 'USA');

-- Equivalent to (more verbose):
SELECT * FROM customers WHERE country = 'Germany' OR country = 'USA';

-- Get customers NOT from Germany, India, or Africa
SELECT * FROM customers
WHERE country NOT IN ('Germany', 'India', 'Africa');
```

Advantage of IN over OR:

- Cleaner, more readable code
- More efficient for multiple values
- Easier to maintain lists of values
- Single condition instead of chained OR statements

Use Case:

- Multiple category filtering
- Exclude specific regions/departments
- Check against predefined lists
- Membership validation

---

5. Pattern Matching - LIKE Operator

Purpose: Search for specific patterns within text/string data

Wildcard Characters:

- % (Percent) - Represents zero, one, or MANY characters
- _ (Underscore) - Represents exactly ONE character

Pattern Rules:

```
%text  = Anything at start, specified characters at end
text%  = Specified characters at start, anything at end
```

```
%text% = Specified characters anywhere in the text
__text = Two characters, then specified text
```

Examples:

```sql
-- Find customers whose first name STARTS with 'M'
SELECT * FROM customers WHERE first_name LIKE 'M%';
/* Matches: Martin, Mary, Mike, etc. */

-- Find customers whose first name ENDS with 'n'
SELECT * FROM customers WHERE first_name LIKE '%n';
/* Matches: Martin, John, Ryan, etc. */

-- Find customers with 'r' ANYWHERE in their name
SELECT * FROM customers WHERE first_name LIKE '%r%';
/* Matches: Martin, Sarah, Robert, etc. */

-- Find customers with 'r' in the 3rd position
SELECT * FROM customers WHERE first_name LIKE '__r%';
/* Matches: Martin (M-a-r), Sarah (S-a-r), etc. */

-- Find customers whose first name does NOT start with 'M'
SELECT * FROM customers WHERE first_name NOT LIKE 'M%';

-- Find customers whose first name does NOT end with 'n'
SELECT * FROM customers WHERE first_name NOT LIKE '%n';
```

Performance Consideration:

- LIKE with leading % (like '%pattern') is slower
- Avoid leading wildcard when possible
- Consider full-text search for large text columns

Use Case:

- Email validation (contains @)
- Phone number pattern matching
- Address searches
- Product code filtering
- Partial name searches

DDL Commands - Database Structure

Definition

DDL (Data Definition Language): Commands that modify the STRUCTURE of databases and tables, NOT the data itself.

Key Characteristic: DDL commands do NOT return data; they change the database schema/structure.

1. CREATE - Creating Tables

Purpose: Define a new table with columns, data types, and constraints

Syntax:

```
CREATE TABLE table_name (
    column_name1 datatype constraint,
    column_name2 datatype constraint,
    CONSTRAINT constraint_name constraint_type (column_name)
);
```

Data Types:

- `INT` - Integer numbers
- `VARCHAR(n)` - Variable length text (max n characters)
- `CHAR(n)` - Fixed length text
- `DATE` - Date values
- `DATETIME` - Date and time values
- `DECIMAL(precision, scale)` - Decimal numbers

Constraints:

- `NOT NULL` - Column must have a value
- `PRIMARY KEY` - Unique identifier for each row
- `UNIQUE` - All values in column must be unique
- `DEFAULT` - Default value if none provided
- `FOREIGN KEY` - Reference to another table

Example:

```
-- Create persons table
CREATE TABLE persons (
    ID INT NOT NULL,
    Person_Name VARCHAR(30) NOT NULL,
    Birth_Date DATE,
    Phone VARCHAR(15) NOT NULL,
    CONSTRAINT pk_key PRIMARY KEY (ID)
);

SELECT * FROM persons;  -- Verify table creation
```

Use Case:

- Setting up new databases
- Creating dimension tables in data warehouse
- Establishing data structure for new applications

2. ALTER - Modifying Table Structure

Purpose: Modify existing table definition (add/remove columns, change data types, add constraints)

Syntax - Add Columns:

```
ALTER TABLE table_name ADD column_name datatype [constraint];
```

Syntax - Drop Columns:

```
ALTER TABLE table_name DROP COLUMN column_name;
```

⚠ **Important Rules**:

- To add MULTIPLE columns in one statement, use ONE ADD keyword only
- Multiple ADD keywords will cause an error
- To add columns in between existing columns, must recreate the table
- ALTER TABLE adds new columns to the END of the table by default

Examples:

```
-- Add multiple columns in one ALTER statement (only 1 ADD keyword)
ALTER TABLE persons
ADD pincode CHAR(6) NOT NULL,
    email VARCHAR(50) NOT NULL;

SELECT * FROM persons;  -- Verify columns added

-- Drop columns
ALTER TABLE persons
DROP COLUMN pincode, email;

-- Correct way for multiple columns:
ALTER TABLE persons
ADD pincode CHAR(6) NOT NULL,
    email VARCHAR(50) NOT NULL;

-- Wrong way (will give error):
ALTER TABLE persons
ADD pincode CHAR(6) NOT NULL;
ADD email VARCHAR(50) NOT NULL;  -- ERROR!
```

Limitations:

- New columns appear at end of table
- Cannot specify column position during ALTER

- To reposition columns, must recreate entire table

Use Case:

- Adding new fields to existing tables
- Removing obsolete columns
- Expanding data collection requirements
- Schema evolution during development

---

3. DROP - Deleting Tables

Purpose: Delete an entire table, column, or database

Syntax:

```
DROP TABLE table_name;
DROP DATABASE database_name;
DROP COLUMN column_name;  -- Within ALTER TABLE context
```

⚠ **Warning**: DROP is PERMANENT and cannot be undone!

Example:

```
-- Delete entire persons table
DROP TABLE persons;
```

Use Case:

- Removing unused tables
- Cleaning up test/temporary structures
- Database reorganization

DML Commands - Data Manipulation

Definition

DML (Data Manipulation Language): Commands that modify DATA within tables (INSERT, UPDATE, DELETE), not the table structure.

Characteristic: DML operations can be rolled back within a transaction.

---

1. INSERT - Adding Data

Purpose: Add new rows of data into a table

Syntax:

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```

Important Rules:

- If column names are NOT specified, values must be provided for ALL columns
- If column names ARE specified, only those columns need values
- Columns with DEFAULT or NULL constraints can be omitted if using column list

Basic INSERT Example:

```
SELECT * FROM customers;  -- Check before insert

-- Insert with specific columns (order doesn't matter)
INSERT INTO customers (id, first_name, country, score)
VALUES
    (6, 'Shubham Sharma', 'India', 580),
    (7, 'Nitin Sharma', 'India', 680);

-- Insert with fewer columns (assumes other columns have defaults or NULL)
INSERT INTO customers (id, first_name, country)
VALUES (8, 'Mohit Sharma', 'India');
```

Use Case:

- Daily data imports
- User registration
- Order entry
- Batch data loading

---

2. INSERT INTO SELECT - Copying Data Between Tables

Purpose: Copy filtered data from source table into target table

Syntax:

```
INSERT INTO target_table (col1, col2, col3)
SELECT col1, col2, col3 FROM source_table WHERE condition;
```

SELECT Clause Options:

- Column names: `SELECT column_name`
- All columns: `SELECT table.*`
- Expressions: Constants like `NULL`, `'UNKNOWN'`, functions, arithmetic

Example:

```
-- First, check what data we're copying
SELECT id, first_name, NULL, 'UNKNOWN'
FROM customers;

-- Insert data from customers into persons table
INSERT INTO persons (id, person_name, birth_date, phone)
SELECT
    id,
    first_name,
    NULL,  -- No birth_date data available
    'UNKNOWN'  -- Placeholder for phone
FROM customers;

-- Verify insertion
SELECT * FROM persons;

/* Explanation:
   - id and first_name are copied from customers table
   - birth_date column filled with NULL (no data available)
   - phone column filled with literal string 'UNKNOWN'
   - All rows from customers are copied into persons */
```

Use Case:

- Data migration between tables
- Creating backups
- Archiving old data
- Transforming data during loading
- Combining data from multiple sources

---

3. UPDATE - Modifying Existing Data

Purpose: Change existing values in a table

Syntax:

```
UPDATE table_name
SET column1 = new_value1, column2 = new_value2
WHERE condition;  /* CRITICAL: Always use WHERE to avoid updating all rows! */
```

⚠ **CRITICAL TIP**:

> Always include a WHERE clause in UPDATE statements! Without WHERE, ALL rows in the table will be updated!

Examples:

```sql
-- Check data before update
SELECT * FROM customers;

-- Update score for customer with id = 6
UPDATE customers
SET score = 0
WHERE id = 6;

-- Verify update
SELECT id, score FROM customers WHERE id = 6;

-- Update multiple columns for one customer
UPDATE customers
SET score = 0, country = 'UK'
WHERE id = 8;

-- Update all rows with NULL score to 0
UPDATE customers
SET score = 0
WHERE score IS NULL;
```

Use Case:

- Correcting data errors
- Bulk status changes
- Applying calculated updates
- Data normalization

---

4. DELETE - Removing Rows

Purpose: Remove one or more rows from a table

Syntax:

```sql
DELETE FROM table_name WHERE condition;
```

⚠ **Warning**:

- DELETE without WHERE removes ALL rows!
- Unlike DROP (removes table structure), DELETE only removes data
- The table structure remains intact

Example:

```sql
-- Delete customers with id > 5
DELETE FROM customers WHERE id > 5;
```

```
-- Verify deletion
SELECT * FROM customers WHERE id > 5;  -- Should return no results if successful
```

Use Case:

- Removing duplicate records
- Deleting outdated records
- Cleaning up test data

---

5. TRUNCATE - Removing All Data

Purpose: Delete ALL content from a table in one operation (faster than DELETE)

Syntax:

```
TRUNCATE TABLE table_name;
```

Difference from DELETE:

- **DELETE**: Can delete specific rows with WHERE; slower for large tables
- **TRUNCATE**: Deletes ALL rows at once; very fast; no WHERE clause available

Example:

```
TRUNCATE TABLE persons;
```

Use Case:

- Clearing temporary/staging tables
- Resetting tables for testing
- Bulk deletion of all data when speed is critical

JOINS - Combining Tables

Definition

JOINS: Combine columns from multiple tables based on related columns (keys)

Key Concept: To join tables, you need a common column (usually a foreign key - primary key relationship) between them.

When to Use Each JOIN Type

Use Case	JOIN Type

Use Case	JOIN Type
Only matching rows	INNER JOIN
All rows from left table + matching right	LEFT JOIN
All rows from right table + matching left	RIGHT JOIN
All rows from both tables	FULL JOIN
Unmatched rows from left only	LEFT ANTI JOIN
Unmatched rows from both	FULL ANTI JOIN
Every combination of rows	CROSS JOIN

Pro Tips

- **Always use LEFT JOIN instead of RIGHT JOIN** - Place the primary table on left, it's clearer
- **Avoid RIGHT JOIN** - Can achieve same results with LEFT JOIN by swapping table positions
- Just keep table name placement in mind when switching between LEFT and RIGHT

---

1. NO JOIN - Separate Result Sets

Purpose: Return data from tables without combining them

Example:

```
-- Returns two separate result sets
SELECT * FROM customers;
SELECT * FROM orders;
```

---

2. INNER JOIN - Only Matching Rows

Purpose: Return ONLY rows where the join condition is TRUE in both tables (overlapping data only)

Syntax:

```
SELECT col1, col2, col3
FROM table_a AS a
INNER JOIN table_b AS b
ON a.key_col = b.key_col;
```

Characteristic: Rows without a match in either table are excluded

Uses:

- Data enrichment (combine related information)

- Data combination (merge multiple tables)
- Filter verification (find existing relationships)

Example:

```
-- Get all customers along with their orders (only customers who ordered)
SELECT
    a.id,
    a.first_name,
    a.country,
    a.score,
    b.sales
FROM customers AS a
INNER JOIN orders AS b
ON a.id = b.customer_id
WHERE b.sales != 0;
```

Visual Representation:

```
Table A     Table B
[1,2,3] ∩ [2,3,4] = [2,3]
    ↑           ↑
Only common values returned
```

---

3. LEFT JOIN - All Left Rows + Matching Right

Purpose: Return ALL rows from LEFT table + matching rows from RIGHT table. Non-matching right rows show NULL.

Syntax:

```
SELECT col1, col2, col3
FROM table_a AS a
LEFT JOIN table_b AS b
ON a.key_col = b.key_col;
```

Characteristics:

- Left table is PRIMARY source (all its rows included)
- Right table is SECONDARY source (only matches included)
- Right table columns show NULL for non-matching rows

Examples:

```sql
-- Get all customers with their orders (including those who didn't order)
SELECT
    a.id,
    a.first_name,
    b.sales
FROM customers AS a
LEFT JOIN orders AS b
ON a.id = b.customer_id;

-- Get only customers who DIDN'T order (anti-join pattern)
SELECT
    a.id,
    a.first_name,
    b.sales
FROM customers AS a
LEFT JOIN orders AS b
ON a.id = b.customer_id
WHERE b.customer_id IS NULL;
/* NULL in b.customer_id means no matching order found */
```

Visual Representation:

```
Table A     Table B
[1,2,3] ← [2,3,4]
Returns all A rows + matching B data
```

---

4. RIGHT JOIN - All Right Rows + Matching Left

Purpose: Return ALL rows from RIGHT table + matching rows from LEFT table

Syntax:

```sql
SELECT col1, col2, col3
FROM table_a AS a
RIGHT JOIN table_b AS b
ON a.key_col = b.key_col;
```

⚠ **Note**: Avoid RIGHT JOIN. Achieve same result with LEFT JOIN by swapping tables.

Example with RIGHT JOIN:

```sql
-- Get all orders with their customer details
SELECT
    a.customer_id,
    a.sales,
```

```
    b.first_name,
    b.country,
    b.score
FROM orders AS a
RIGHT JOIN customers AS b
ON a.customer_id = b.id;
```

Equivalent with LEFT JOIN (PREFERRED):

```
-- Same result, clearer approach
SELECT
    a.customer_id,
    a.sales,
    b.first_name,
    b.country,
    b.score
FROM customers AS b
LEFT JOIN orders AS a
ON b.id = a.customer_id;
```

---

5. FULL JOIN - All Rows from Both Tables

Purpose: Return ALL rows from BOTH tables. NULL appears where matches don't exist.

Syntax:

```
SELECT col1, col2, col3
FROM table_a AS a
FULL JOIN table_b AS b
ON a.key_col = b.key_col;
```

Example:

```
-- Get all data from customers and orders
SELECT *
FROM customers AS a
FULL JOIN orders AS b
ON a.id = b.customer_id;
```

Visual Representation:

```
Table A     Table B
[1,2,3] ∪ [2,3,4] = [1,2,3,4]
All values from both tables
```

---

6. LEFT ANTI JOIN - Unmatched Left Rows Only

Purpose: Return rows from LEFT table that have NO match in RIGHT table (opposite of INNER JOIN)

Use Case: Find missing data or unmatched records

Implementation:

```
SELECT *
FROM customers AS a
LEFT JOIN orders AS b
ON a.id = b.customer_id
WHERE b.customer_id IS NULL;
/* NULL in RIGHT table column = no match found */
```

Example:

```
-- Find customers who haven't placed orders
SELECT *
FROM customers AS a
LEFT JOIN orders AS b
ON a.id = b.customer_id
WHERE b.customer_id IS NULL;
```

---

7. RIGHT ANTI JOIN - Unmatched Right Rows Only

Purpose: Return rows from RIGHT table that have NO match in LEFT table

Example with RIGHT JOIN:

```
-- Find orders without matching customers
SELECT *
FROM customers AS a
RIGHT JOIN orders AS b
ON a.id = b.customer_id
WHERE a.id IS NULL;
```

Preferred Implementation (using LEFT ANTI):

```
-- Same result, clearer logic
SELECT *
FROM orders AS a
LEFT JOIN customers AS b
```

```
ON b.id = a.customer_id
WHERE b.id IS NULL;
```

---

8. FULL ANTI JOIN - Unmatched from Both Tables

Purpose: Return rows that don't match in EITHER table

Syntax:

```
SELECT *
FROM table_a AS a
FULL JOIN table_b AS b
ON a.key_col = b.key_col
WHERE a.key_col IS NULL OR b.key_col IS NULL;
```

Example:

```
-- Find customers without orders AND orders without customers
SELECT *
FROM customers AS a
FULL JOIN orders AS b
ON a.id = b.customer_id
WHERE b.customer_id IS NULL OR a.id IS NULL;
/* Note: OR not AND (both can't be true simultaneously) */
```

---

9. CROSS JOIN - All Possible Combinations

Purpose: Combine every row from left table with every row from right table

Result Size: Rows in A × Rows in B

Syntax:

```
SELECT *
FROM table_a
CROSS JOIN table_b;
```

Example:

```
-- Generate all possible combinations of customers and orders
SELECT *
FROM customers
CROSS JOIN orders;
```

⚠ **Use Carefully**: Results can be VERY large (exponential growth)

Use Case:

- Generating date ranges
- Creating combinations for cartesian products
- Test data generation

---

10. Joining Multiple Tables

Purpose: Combine data from 3+ tables in single query

Strategy:

- Identify master table (primary data source)
- Join secondary tables one at a time for additional information
- Use LEFT JOIN for each additional table (preserves master table rows)

Syntax:

```
SELECT col1, col2, col3
FROM master_table AS m
LEFT JOIN secondary_table1 AS s1 ON m.key = s1.key
LEFT JOIN secondary_table2 AS s2 ON m.key = s2.key
LEFT JOIN secondary_table3 AS s3 ON m.key = s3.key;
```

Example:

```
-- Retrieve orders with customer, product, and employee details
USE SalesDB;

SELECT
    o.OrderID,
    o.Sales,
    o.CustomerID,
    c.FirstName,
    c.LastName,
    c.Country,
    c.Score,
    p.Product,
    p.Category,
    e.EmployeeID AS SalesPersonID,
    e.FirstName AS SalesPersonName,
    e.ManagerID
FROM Sales.Orders AS o
LEFT JOIN Sales.Customers AS c ON o.CustomerID = c.CustomerID
LEFT JOIN Sales.Products AS p ON o.ProductID = p.ProductID
LEFT JOIN Sales.Employees AS e ON e.EmployeeID = o.SalesPersonID;
```

```
/* Execution Flow:
   1. START: Orders table (master)
   2. Add customer details (left join on customer_id)
   3. Add product details (left join on product_id)
   4. Add employee details (left join on employee_id)
   5. RESULT: Single table with all related information */
```

SET Operators - Row Combination

Definition

SET Operators: Combine rows from multiple queries (not columns like JOINS)

Key Difference from JOINS:

- **JOINS**: Combine COLUMNS from tables side-by-side
- **SET Operators**: Stack ROWS from tables on top of each other

Rules for SET Operators

Critical Rules:

1. Exact same number of columns in each query
2. Compatible/matching data types in corresponding columns
3. Same order of columns in both queries
4. **Column names from FIRST query control result column names** (second query names ignored)
5. ORDER BY used only ONCE at the very end (optional)

---

1. UNION - Distinct Rows Only

Purpose: Combine rows from two queries, removing duplicate rows

Syntax:

```
SELECT col1, col2 FROM table1
UNION
SELECT col1, col2 FROM table2
ORDER BY col1;  /* Optional, must be at end */
```

Characteristics:

- Removes duplicate rows automatically
- Slower than UNION ALL (performs deduplication)
- Each row appears only once in results

Example:

```
-- Combine customer and employee names (remove duplicates)
SELECT
    FirstName,
    LastName
FROM Sales.Customers
UNION
SELECT
    FirstName,
    LastName
FROM Sales.Employees
ORDER BY FirstName ASC;
/* If John Doe appears in both tables, shows only once */
```

---

2. UNION ALL - Include All Rows (with Duplicates)

Purpose: Combine rows keeping all rows including duplicates

Syntax:

```
SELECT col1, col2 FROM table1
UNION ALL
SELECT col1, col2 FROM table2;
```

Characteristics:

- Faster than UNION (no deduplication)
- Useful for finding duplicates/quality issues
- All rows appear even if duplicated

Example:

```
-- Combine customers and employees (include all, even if names appear twice)
SELECT
    CustomerID,
    FirstName,
    LastName
FROM Sales.Customers
UNION ALL
SELECT
    EmployeeID,
    FirstName,
    LastName
FROM Sales.Employees;
```

---

3. EXCEPT (MINUS) - Rows in First but Not Second

Purpose: Return rows from first query that DON'T appear in second query (acts as filter)

Syntax:

```
SELECT col1, col2 FROM table1
EXCEPT
SELECT col1, col2 FROM table2;
```

Characteristics:

- Second query acts as filter only (its rows never appear in results)
- Order of queries MATTERS (not commutative)
- Returns only distinct rows from first query
- Similar to LEFT ANTI JOIN

Example:

```
-- Find employees who are NOT customers
SELECT
    EmployeeID,
    FirstName,
    LastName
FROM Sales.Employees
EXCEPT
SELECT
    CustomerID,
    FirstName,
    LastName
FROM Sales.Customers;
/* Result: Employees with names not found in customers table */
```

---

4. INTERSECT - Common Rows in Both Queries

Purpose: Return only rows that appear in BOTH queries

Syntax:

```
SELECT col1, col2 FROM table1
INTERSECT
SELECT col1, col2 FROM table2;
```

Characteristics:

- Returns only rows found in both queries
- Order doesn't matter (commutative)
- Similar to INNER JOIN

- Returns only distinct rows

Example:

```
-- Find employees who are also customers
SELECT
    EmployeeID,
    FirstName,
    LastName
FROM Sales.Employees
INTERSECT
SELECT
    CustomerID,
    FirstName,
    LastName
FROM Sales.Customers;
/* Result: Names that appear in both tables */
```

SET Operators Use Cases

1. Combining Similar Data (UNION)

Scenario: Need to report on INDIVIDUALS (could be customers, employees, suppliers, or students)

Old Approach:

```
-- Multiple separate queries (untidy report)
SELECT ... FROM customers;
SELECT ... FROM employees;
SELECT ... FROM suppliers;
```

Better Approach with UNION:

```
-- Single unified table of all individuals
SELECT CustomerID AS IndividualID, FirstName, LastName, 'Customer' AS Type
FROM customers
UNION
SELECT EmployeeID, FirstName, LastName, 'Employee'
FROM employees
UNION
SELECT SupplierID, FirstName, LastName, 'Supplier'
FROM suppliers;
```

---

2. Delta Detection (EXCEPT)

Scenario: Identify NEW data between data loads

Use Case:

- Data pipelines load daily data
- Day 1 data already in warehouse
- Day 2 new data received (some duplicates, some new)
- Use EXCEPT to find ONLY new records

Example:

```
-- Find new customers from today's load (not in yesterday's load)
SELECT CustomerID, Email, LoadDate
FROM NewCustomers_Today
EXCEPT
SELECT CustomerID, Email, LoadDate
FROM Customers_Warehouse;
```

---

3. Data Completeness Check (EXCEPT)

Scenario: Verify data migration between databases

Process:

1. Migrate data from DB A to DB B
2. Check if all data transferred: Run EXCEPT query
3. If NO results → All data transferred successfully
4. If results → Records missing in destination DB

Example:

```
-- Check if all employees transferred from DB_A to DB_B
SELECT * FROM DB_A.Employees
EXCEPT
SELECT * FROM DB_B.Employees;
-- If empty result = complete transfer; If rows = missing data
```

SET Operators Summary

Operator	Purpose	Result	Speed
UNION	Combine + deduplicate	Distinct rows	Slower
UNION ALL	Combine as-is	All rows with duplicates	Faster
EXCEPT	Rows in A not B	Filtering results	Medium

Operator	Purpose	Result	Speed
INTERSECT	Rows in both A & B	Common rows	Medium

String Functions - Text Manipulation

Definition

String functions manipulate text data, perform calculations on text length, extract portions, and transform case.

Categories of String Functions

---

1. Text Manipulation Functions

CONCAT - Combining Multiple Strings

Purpose: Combine multiple strings into one

Syntax:

```
SELECT CONCAT(str1, separator, str2, separator, str3);
```

Example:

```
-- Combine first name and country
SELECT
    FirstName,
    Country,
    CONCAT(FirstName, '-', Country) AS Name_Country
FROM Sales.Customers;
/* Results like: John-USA, Sarah-Germany */
```

UPPER / LOWER - Case Conversion

Purpose: Convert text to all UPPERCASE or all lowercase

Syntax:

```
SELECT UPPER(column_name) AS uppercase_version;
SELECT LOWER(column_name) AS lowercase_version;
```

Example:

```sql
-- Convert customer names to different cases
SELECT
    FirstName,
    LOWER(FirstName) AS firstname_lower,
    UPPER(FirstName) AS firstname_upper
FROM Sales.Customers;
```

Use Case:

- Standardizing data format
- Case-insensitive comparisons
- Report formatting

TRIM - Removing Whitespace

Purpose: Remove leading (start) and trailing (end) whitespace/spaces

Syntax:

```sql
SELECT TRIM(column_name);
```

Example:

```sql
-- Find customers with leading/trailing spaces
SELECT
    FirstName
FROM Sales.Customers
WHERE FirstName != TRIM(FirstName);

-- Show spaces removed
SELECT
    FirstName,
    LEN(FirstName) AS Original_Length,
    LEN(TRIM(FirstName)) AS Trimmed_Length,
    LEN(FirstName) - LEN(TRIM(FirstName)) AS Spaces
FROM Sales.Customers
WHERE LEN(FirstName) != LEN(TRIM(FirstName));
```

Use Case:

- Data cleaning (remove accidental spaces)
- Standardizing input
- Quality assurance

REPLACE - Substituting Characters

Purpose: Replace specific characters/strings with new characters

Syntax:

```
SELECT REPLACE(original_string, old_value, new_value);
```

Example:

```
-- Remove dashes from phone number
SELECT
    '123-456-7890' AS Original_Phone,
    REPLACE('123-456-7890', '-', '') AS Clean_Phone;
/* Result: 123-456-7890 → 1234567890 */

-- Replace dashes with slashes
SELECT REPLACE('123-456-7890', '-', '/') AS formatted_phone;
/* Result: 123/456/7890 */

-- Change file extension
SELECT REPLACE('report.txt', '.txt', '.csv') AS new_filename;
/* Result: report.csv */
```

Use Case:

- Formatting phone numbers
- Converting file extensions
- Standardizing separators
- Data normalization

---

2. Text Length Calculation

LEN - Character Count

Purpose: Count the number of characters in a string (including spaces, digits, special characters)

Syntax:

```
SELECT LEN(column_name);
```

Example:

```sql
-- Count characters in customer names
SELECT
    FirstName,
    LEN(FirstName) AS Name_Length
FROM Sales.Customers;

-- Count characters in dates
SELECT
    '2025-08-30' AS Date_Value,
    LEN('2025-08-30') AS Date_Length;  -- Result: 10 (includes dashes)
```

Use Case:

- Validation (check minimum/maximum length)
- Data quality checks
- String analysis

---

3. Text Extraction Functions

LEFT - Extract from Start

Purpose: Extract a specific number of characters from the LEFT (start) of a string

Syntax:

```sql
SELECT LEFT(string, number_of_chars);
```

Example:

```sql
-- Extract first 3 characters of customer names
SELECT
    FirstName,
    LEFT(FirstName, 3) AS Short_Name
FROM Sales.Customers;
/* John → Joh, Sarah → Sar */
```

RIGHT - Extract from End

Purpose: Extract a specific number of characters from the RIGHT (end) of a string

Syntax:

```sql
SELECT RIGHT(string, number_of_chars);
```

Example:

```
-- Extract last 3 characters of last names
SELECT
    LastName,
    RIGHT(LastName, 3) AS Last_3_Chars
FROM Sales.Customers;
/* Johnson → son, Williams → ams */
```

SUBSTRING - Extract Middle/Specific Position

Purpose: Extract characters from a specific position for a specific length

Syntax:

```
SELECT SUBSTRING(string, start_position, length);
```

Parameters:

- **start_position**: Where to begin (1 = first character)
- **length**: How many characters to extract

Examples:

```
-- Extract 2 characters starting at position 2
SELECT
    FirstName,
    SUBSTRING(FirstName, 2, 2) AS Middle_2_Chars
FROM Sales.Customers;
/* John → oh, Sarah → ar */

-- Extract all characters after position 2
SELECT
    FirstName,
    SUBSTRING(FirstName, 2, LEN(FirstName)) AS From_2nd_Char
FROM Sales.Customers;
/* John → ohn, Sarah → arah */

-- Extract after position 2 without spaces
SELECT
    FirstName,
    SUBSTRING(TRIM(FirstName), 2, LEN(FirstName)) AS Trimmed_Substring
FROM Sales.Customers;
```

Use Case:

- Extracting area codes from phone
- Getting year/month from formatted dates
- Parsing fixed-width data
- Code/ID parsing

Number Functions - Numeric Operations

1. ROUND - Rounding Values

Purpose: Round a number to a specified number of decimal places

Syntax:

```
SELECT ROUND(number, decimal_places);
```

Logic:

- If digit after desired position is >= 5: round UP
- If digit after desired position is < 5: round DOWN

Example:

```
-- Round to 2 decimal places
SELECT
    3.516,
    ROUND(3.516, 2) AS Rounded_2Places;  /* 3.52 */

-- Round to 1 decimal place
SELECT 3.516, ROUND(3.516, 1) AS Rounded_1Place;  /* 3.5 */

-- Round to 0 decimal places (whole number)
SELECT 3.516, ROUND(3.516, 0) AS Rounded_Whole;  /* 4 */
```

Use Case:

- Financial calculations (currency precision)
- Statistical reporting
- Simplifying decimals

---

2. ABS - Absolute Value

Purpose: Convert negative numbers to positive (remove negative sign)

Syntax:

```
SELECT ABS(number);
```

Example:

```
-- Convert negative values to positive
SELECT -10, ABS(-10);  /* Result: 10 */

-- Fix negative sales due to calculation errors
SELECT
    OrderID,
    Sales,
    ABS(Sales) AS Sales_Positive
FROM Sales.Orders
WHERE Sales < 0;
```

Use Case:

- Fixing data entry errors (negative amounts)
- Distance calculations
- Financial analysis

Date & Time Functions

Date & Time Formats in SQL Server

- **Date format**: YYYY-MM-DD (ISO 8601 standard)
- **Time format**: HH:MM:SS (24-hour format)
- **DateTime**: YYYY-MM-DD HH:MM:SS

Date Data Sources

1. **Stored in database**: Historical dates from tables
2. **Hardcoded constants**: Specific date strings in query
3. **System function**: Current date/time using GETDATE()

1. GETDATE() - Current Date/Time

Purpose: Return current date and time when query executes

Syntax:

```
SELECT GETDATE();
```

Example:

```sql
-- Get current date and time
SELECT GETDATE();

-- Add current timestamp to orders
SELECT
    OrderID,
    OrderDate,
    ShipDate,
    CreationTime,
    GETDATE() AS Today
FROM Sales.Orders;
```

Use Case:

- Timestamp creation
- Audit trails
- Age/duration calculations

---

2. Part Extraction Functions

DAY / MONTH / YEAR

Purpose: Extract specific parts of a date as numbers

Syntax:

```sql
SELECT
    DAY(date_column) AS day_number,       -- Returns 1-31
    MONTH(date_column) AS month_number,   -- Returns 1-12
    YEAR(date_column) AS year_number;     -- Returns year like 2025
```

Example:

```sql
-- Extract year, month, day from creation date
SELECT
    OrderID,
    CreationTime,
    YEAR(CreationTime) AS Year_Created,
    MONTH(CreationTime) AS Month_Created,
    DAY(CreationTime) AS Day_Created,
    OrderDate
FROM Sales.Orders;
```

DATEPART - Flexible Part Extraction

Purpose: Extract any part of a date (year, month, day, hour, minute, week, quarter, etc.)

Syntax:

```
SELECT DATEPART(part_abbreviation, date_column);
```

Common Abbreviations:

- `yy` = Year
- `MM` = Month
- `DD` = Day
- `HH` = Hour
- `MI` or `MM` = Minute (use MI to avoid conflict with month)
- `SS` = Second
- `WK` = Week
- `QQ` = Quarter

Examples:

```
-- Extract various parts using abbreviations
SELECT
    OrderID,
    CreationTime,
    DATEPART(yy, CreationTime) AS Year,
    DATEPART(MM, CreationTime) AS Month,
    DATEPART(DD, CreationTime) AS Day,
    DATEPART(HH, CreationTime) AS Hour,
    DATEPART(MI, CreationTime) AS Minute,
    DATEPART(SS, CreationTime) AS Second
FROM Sales.Orders;

-- Or use full names (also valid)
SELECT
    OrderID,
    CreationTime,
    DATEPART(YEAR, CreationTime) AS Year,
    DATEPART(MONTH, CreationTime) AS Month,
    DATEPART(QUARTER, CreationTime) AS Quarter
FROM Sales.Orders;

-- Extract week and weekday
SELECT
    OrderID,
    CreationTime,
    DATEPART(WEEK, CreationTime) AS Week_Number,
    DATEPART(WEEKDAY, CreationTime) AS Weekday
FROM Sales.Orders;
```

Use Case:

- Extracting quarters for financial analysis
- Finding week numbers for scheduling
- Hour extraction for time-of-day analysis

DATENAME - Get Name Instead of Number

Purpose: Return the NAME of the date part (e.g., "January" instead of 1, "Monday" instead of 2)

Syntax:

```
SELECT DATENAME(part, date_column);
```

Example:

```
-- Get month and day names
SELECT
    OrderID,
    CreationTime,
    DATENAME(MM, CreationTime) AS Month_Name,
    DATENAME(weekday, CreationTime) AS Day_Name
FROM Sales.Orders;
/* Results: January, February, ... December, Monday, Tuesday, ... Sunday */
```

DATETRUNC - Truncate to Specific Part

Purpose: Keep only specified part of date, reset rest to default values (00 for time, 01 for date)

Syntax:

```
SELECT DATETRUNC(part, date_column);
```

Default Reset Values:

- Time parts reset to: 00:00:00
- Date parts reset to: 01

Examples:

```
-- Truncate to different parts
/* Input: 2025-08-20 18:55:45 */

SELECT
```

```
    DATETRUNC(YEAR, CreationTime),    /* 2025-01-01 00:00:00 */
    DATETRUNC(MONTH, CreationTime),   /* 2025-08-01 00:00:00 */
    DATETRUNC(DAY, CreationTime),     /* 2025-08-20 00:00:00 */
    DATETRUNC(HOUR, CreationTime),    /* 2025-08-20 18:00:00 */
    DATETRUNC(MINUTE, CreationTime)   /* 2025-08-20 18:55:00 */
FROM Sales.Orders;
```

Practical Use:

```
-- Group orders by month (all times normalized to start of month)
SELECT
    DATETRUNC(MONTH, CreationTime) AS Month_Start,
    COUNT(OrderID) AS Total_Orders
FROM Sales.Orders
GROUP BY DATETRUNC(MONTH, CreationTime);
```

EOMONTH - End of Month

Purpose: Get the last day of the month for a given date

Syntax:

```
SELECT EOMONTH(date_column);
```

Example:

```
-- Get end date of each order's month
SELECT
    OrderID,
    DATETRUNC(DAY, CreationTime) AS Month_Start,
    EOMONTH(CreationTime) AS Month_End
FROM Sales.Orders;
```

---

3. Date Formatting and Conversion

FORMAT - Custom Date/Number Formatting

Purpose: Format dates and numbers into readable strings with custom patterns

Syntax:

```
SELECT FORMAT(value, 'format_pattern' [, culture]);
```

Common Date Patterns:

- `'dd-MM-yy'` = 20-08-25 (European)
- `'MM-dd-yy'` = 08-20-25 (US)
- `'dd/MM/yyyy'` = 20/08/2025
- `'MMM yy'` = Aug 25 (abbreviated month)
- `'MMMM dd, yyyy'` = August 20, 2025 (full format)
- `'dddd'` = Wednesday (full day name)
- `'ddd'` = Wed (abbreviated day name)

Examples:

```
-- Format dates in different styles
SELECT
    OrderID,
    CreationTime,
    FORMAT(CreationTime, 'dd-MM-yy') AS European_Format,
    FORMAT(CreationTime, 'MM-dd-yy') AS US_Format,
    FORMAT(CreationTime, 'dd') AS Day_Only,
    FORMAT(CreationTime, 'ddd') AS Abbreviated_Day,
    FORMAT(CreationTime, 'dddd') AS Full_Day_Name,
    FORMAT(CreationTime, 'MM') AS Month_Number,
    FORMAT(CreationTime, 'MMM') AS Month_Abbr,
    FORMAT(CreationTime, 'MMMM') AS Full_Month_Name
FROM Sales.Orders;

-- Create custom format combining parts
SELECT
    OrderID,
    'Day ' + FORMAT(CreationTime, 'ddd MMM') +
    ' Q' + DATENAME(QUARTER, CreationTime) + ' ' +
    FORMAT(CreationTime, 'yy HH:mm:ss tt') AS Custom_Format
FROM Sales.Orders;
/* Example result: Day Wed Aug Q3 25 02:34:56 PM */
```

Number Formatting:

```
SELECT
    1234567.89,
    FORMAT(1234567.89, 'N') AS With_Thousands,     /* 1,234,567.89 */
    FORMAT(1234567.89, 'C') AS Currency,           /* $1,234,567.89 */
    FORMAT(0.567, 'P') AS Percentage;              /* 56.70% */
```

---

4. Date Arithmetic

DATEADD - Adding/Subtracting Time

Purpose: Add or subtract specific time intervals from a date

Syntax:

```
SELECT DATEADD(interval_part, number, date_column);
```

Intervals:

- `YEAR` = Years
- `MONTH` = Months
- `DAY` = Days
- `HOUR` = Hours
- `MINUTE` = Minutes
- `SECOND` = Seconds

Examples:

```
-- Add/subtract time from dates
SELECT
    OrderID,
    OrderDate,
    DATEADD(YEAR, 2, OrderDate) AS Two_Years_Later,
    DATEADD(MONTH, -4, OrderDate) AS Four_Months_Earlier,
    DATEADD(DAY, -10, OrderDate) AS Ten_Days_Earlier
FROM Sales.Orders;
```

DATEDIFF - Calculating Differences

Purpose: Find the difference between two dates

Syntax:

```
SELECT DATEDIFF(interval_part, start_date, end_date);
```

Examples:

```
-- Find days to ship for each order
SELECT
    FORMAT(OrderDate, 'MMM yyyy') AS Order_Month,
    AVG(DATEDIFF(DD, OrderDate, ShipDate)) AS Avg_Delivery_Days
FROM Sales.Orders
GROUP BY FORMAT(OrderDate, 'MMM yyyy');

-- Calculate employee age
```

```
SELECT
    EmployeeID,
    FirstName,
    DATEDIFF(yyyy, BirthDate, GETDATE()) AS Age
FROM Sales.Employees;

-- Time gap between orders
SELECT
    OrderID,
    OrderDate AS Current_OrderDate,
    LAG(OrderDate) OVER (ORDER BY OrderDate) AS Previous_Order_Date,
    DATEDIFF(DAY, LAG(OrderDate) OVER (ORDER BY OrderDate), OrderDate) AS
Days_Between_Orders
FROM Sales.Orders;
```

ISDATE - Validate Date Values

Purpose: Check if a value is a valid date format

Syntax:

```
SELECT ISDATE(value);
/* Returns 1 = valid date, 0 = invalid */
```

Example:

```
-- Check which formats are valid
SELECT
    ISDATE('123') AS Check1,              /* 0 - Invalid */
    ISDATE('2025-08-20') AS Check2,      /* 1 - Valid */
    ISDATE('20-08-2025') AS Check3,      /* 1 - Valid (location dependent) */
    ISDATE('2025') AS Check4,            /* 0 - Invalid */
    ISDATE('08') AS Check5;              /* 0 - Invalid */

-- Handle invalid dates in subquery
SELECT
    OrderDate,
    ISDATE(OrderDate) AS DateCheck,
    CASE WHEN ISDATE(OrderDate) = 1
        THEN CAST(OrderDate AS date)
        ELSE '2000-01-01'
    END AS Cleaned_Date
FROM (
    SELECT '2025-08-20' AS OrderDate UNION
    SELECT '2025-08-21' UNION
    SELECT '2025-08-23' UNION
    SELECT 'INVALID'  /* Unidentified format */
```

```
) AS t
WHERE ISDATE(OrderDate) = 0;
```

NULL Functions - Handling Missing Data

Definition

NULL: Represents missing, unknown, or undefined data - NOT zero, NOT empty string, NOT blank space

When to Use NULL Functions

Situation	Function to Use
Replace NULL with a value	ISNULL, COALESCE
Replace a value with NULL	NULLIF
Check if data is NULL	IS NULL, IS NOT NULL

---

1. ISNULL - Replace NULL with Value

Purpose: Replace NULL with a specified value

Syntax:

```
SELECT ISNULL(column_name, replacement_value);
```

Limitation:

- Can only handle 2 values (original and replacement)
- If replacement is also NULL, won't work as expected

Example:

```
-- Replace NULL shipping addresses with 'Unknown'
SELECT
    ShippingAddress,
    ISNULL(ShippingAddress, 'Unknown') AS Safe_Address
FROM Orders;
```

---

2. COALESCE - Multiple NULL Replacements

Purpose: Replace NULL by checking multiple columns in order, returning first non-NULL value

Syntax:

```
SELECT COALESCE(col1, col2, col3, 'Default Value');
/* Returns first non-NULL; if all NULL returns default */
```

Advantage over ISNULL:

- Unlimited columns to check
- Can specify multiple fallback options
- Handles cases where replacement might also be NULL

Example:

```
-- Use billing address if shipping is NULL, else use 'N/A'
SELECT
    ShippingAddress,
    BillingAddress,
    COALESCE(ShippingAddress, BillingAddress, 'N/A') AS Final_Address
FROM Orders;

-- Find full address with multiple fallbacks
SELECT
    COALESCE(PreferredAddress, SecondaryAddress, DefaultAddress, 'Address TBD') AS
Address_To_Use
FROM Customers;
```

---

3. NULLIF - Convert Value to NULL

Purpose: Return NULL if two values are equal; otherwise return first value

Syntax:

```
SELECT NULLIF(value1, value2);
```

Use Case - Prevent Division by Zero:

```
-- Dangerous: Divides by zero if quantity = 0
SELECT
    OrderID,
    Sales / Quantity AS Price_Per_Unit
FROM Sales.Orders;
/* ERROR if Quantity = 0 */

-- Safe: NULLIF converts 0 to NULL, avoiding error
SELECT
    OrderID,
    Sales / NULLIF(Quantity, 0) AS Price_Per_Unit
```

```
FROM Sales.Orders;
/* Returns NULL instead of error when Quantity = 0 */
```

---

4. IS NULL / IS NOT NULL - Check for NULL

Purpose: Test whether a value IS NULL or IS NOT NULL

Syntax:

```
SELECT * FROM table WHERE column IS NULL;
SELECT * FROM table WHERE column IS NOT NULL;
```

Example:

```
-- Find customers with no score assigned
SELECT * FROM Sales.Customers WHERE Score IS NULL;

-- Find customers with a score
SELECT * FROM Sales.Customers WHERE Score IS NOT NULL;

-- Use in anti-join pattern
SELECT *
FROM customers AS c
LEFT JOIN orders AS o ON c.CustomerID = o.CustomerID
WHERE o.CustomerID IS NULL;  /* Customers with no orders */
```

NULL Handling in Aggregations

Important Behavior:

- Aggregate functions (SUM, AVG, MIN, MAX) **IGNORE NULL values**
- **Exception**: COUNT(*) counts NULL rows, but COUNT(column) ignores NULLs

Example:

```
-- NULL ignored in AVG - may skew calculations
SELECT
    CustomerID,
    Score,
    AVG(Score) OVER() AS AvgScores
FROM Sales.Customers;
/* NULL scores ignored in average */

-- Replace NULLs BEFORE aggregation for accurate results
SELECT
```

```
    CustomerID,
    Score,
    AVG(COALESCE(Score, 0)) OVER() AS AvgScores_With_Replacements
FROM Sales.Customers;
/* NULL replaced with 0, included in average */
```

NULL vs EMPTY vs BLANK SPACES

Aspect	NULL	EMPTY STRING	BLANK SPACE
Value	Missing/Unknown	Known empty value ''	Known space ' '
Data Type	Special marker	String(0)	String(1+)
Storage	Very minimal	Occupies memory	Occupies memory
Performance	Best	Fast	Slow
Comparison	IS NULL	= ''	= ' '
DATALENGTH	NULL	0	1+

Example - Demonstrating Differences:

```
WITH TestData AS (
    SELECT 1 AS ID, 'A' AS Category UNION
    SELECT 2, NULL UNION
    SELECT 3, '' UNION
    SELECT 4, ' '
)
SELECT *,
    DATALENGTH(Category) AS Data_Length
FROM TestData;

/* Results:
   ID | Category | Data_Length
   1  | A        | 1
   2  | NULL     | NULL (no space)
   3  | (empty)  | 0 (empty string)
   4  | (space)  | 1 (one space) */
```

Data Policies for Handling NULLs

Policy 1: Use Nulls and Trim Spaces (Avoid Blanks)

```
SELECT *,
    DATALENGTH(TRIM(Category)) AS Policy_1
```

```
FROM Orders;
/* Removes leading/trailing spaces */
```

Policy 2: Use Only Nulls (No Empty Strings or Blanks)

```
SELECT *,
    NULLIF(TRIM(Category), '') AS Policy_2
FROM Orders;
/* Replace empty strings with NULL after trimming */
```

Policy 3: Use Default Value (No NULLs)

```
SELECT *,
    COALESCE(NULLIF(TRIM(Category), ''), 'Unknown') AS Policy_3
FROM Orders;
/* Replace NULLs and blanks with 'Unknown' */
```

CASE Statement - Conditional Logic

Definition

CASE Statement: Evaluates conditions and returns values when first condition is TRUE

Purpose: Transform data, categorize values, create conditional columns

Syntax

Standard CASE:

```
SELECT
    CASE
        WHEN condition1 THEN result1
        WHEN condition2 THEN result2
        ...
        ELSE default_result
    END AS new_column_name
FROM table_name;
```

Simple CASE:

```
SELECT
    CASE column_name
        WHEN value1 THEN result1
```

```
        WHEN value2 THEN result2
        ELSE default_result
    END AS new_column_name
FROM table_name;
```

Rules

1. **Data type consistency**: All result values must be same data type
2. **ELSE is optional**: If no condition matches and no ELSE, returns NULL
3. **Order matters**: Evaluates conditions top-to-bottom, stops at first TRUE

Use Case 1: Data Categorization

Purpose: Group data into categories based on conditions

Example:

```
-- Categorize sales into HIGH, MEDIUM, LOW
SELECT
    OrderID,
    Sales,
    CASE
        WHEN Sales > 50 THEN 'HIGH'
        WHEN Sales > 20 AND Sales <= 50 THEN 'MEDIUM'
        ELSE 'LOW'
    END AS Sales_Frequency
FROM Sales.Orders;

-- Then aggregate by category
SELECT
    Sales_Frequency,
    SUM(Sales) AS Total_Sales
FROM (
    SELECT
        Sales,
        CASE
            WHEN Sales > 50 THEN 'HIGH'
            WHEN Sales > 20 AND Sales <= 50 THEN 'MEDIUM'
            ELSE 'LOW'
        END AS Sales_Frequency
    FROM Sales.Orders
) AS t
GROUP BY Sales_Frequency
ORDER BY Total_Sales DESC;
```

Use Case 2: Value Mapping

Purpose: Transform values from one form to another

Example:

```
-- Expand abbreviated gender to full text
SELECT
    EmployeeID,
    FirstName,
    Gender,
    CASE
        WHEN Gender = 'M' THEN 'Male'
        WHEN Gender = 'F' THEN 'Female'
        ELSE 'Unknown'
    END AS Gender_Full
FROM Sales.Employees;

-- Simpler CASE syntax for value mapping
SELECT
    CustomerID,
    FirstName,
    Country,
    CASE Country
        WHEN 'Germany' THEN 'DE'
        WHEN 'USA' THEN 'US'
        WHEN 'India' THEN 'IN'
        ELSE 'N/A'
    END AS Country_Code
FROM Sales.Customers;
```

Use Case 3: Handling NULLs

Purpose: Replace NULL values with specific values

Example:

```
-- Replace NULL scores with 0 in calculations
SELECT
    CustomerID,
    FirstName,
    Score,
    CASE
        WHEN Score IS NULL THEN 0
        ELSE Score
    END AS ScoreClean,
    AVG(CASE
        WHEN Score IS NULL THEN 0
        ELSE Score
    END) OVER() AS AvgScore
FROM Sales.Customers;
```

Use Case 4: Conditional Aggregation

Purpose: Sum/count only rows meeting specific conditions

Example:

```
-- Count orders with sales > 30 per customer
SELECT
    CustomerID,
    SUM(CASE WHEN Sales > 30 THEN 1 ELSE 0 END) AS HighSalesCount
FROM Sales.Orders
GROUP BY CustomerID;

-- Sum sales by category
SELECT
    SUM(CASE WHEN Sales > 50 THEN Sales ELSE 0 END) AS HighSalesTotal,
    SUM(CASE WHEN Sales BETWEEN 20 AND 50 THEN Sales ELSE 0 END) AS
MediumSalesTotal,
    SUM(CASE WHEN Sales < 20 THEN Sales ELSE 0 END) AS LowSalesTotal
FROM Sales.Orders;

-- Count rows meeting multiple criteria
SELECT
    ProductID,
    SUM(CASE WHEN Sales BETWEEN 20 AND 50 THEN 1 ELSE 0 END) AS Orders_20_To_50
FROM Sales.Orders
GROUP BY ProductID;
```

Aggregate Functions - Data Summarization

Definition

Aggregate Functions: Perform calculations across multiple rows, returning single summary values

Key Rules:

> When using aggregate functions in SELECT, every non-aggregated column must appear in GROUP BY clause

Aggregate Functions

COUNT - Row Counting

Purpose: Count rows in a table

Syntax:

```
SELECT COUNT(*) AS TotalRows FROM table_name;    /* Counts all rows, including
NULL */
```

```
SELECT COUNT(column_name) AS NonNullCount FROM table_name;  /* Counts non-NULL
values */
```

Example:

```
-- Count all customers
SELECT COUNT(*) AS TotalCustomers FROM Sales.Customers;

-- Count non-NULL scores (nulls ignored)
SELECT COUNT(Score) FROM Sales.Customers;
```

SUM - Total Sum

Purpose: Calculate total sum of numeric column

Syntax:

```
SELECT SUM(column_name) AS Total FROM table_name;
```

Example:

```
-- Total sales across all orders
SELECT SUM(Sales) AS TotalSales FROM Sales.Orders;
```

AVG - Average Value

Purpose: Calculate average value (ignores NULLs)

Syntax:

```
SELECT
    Country,
    AVG(Score) AS AverageScore
FROM Sales.Customers
GROUP BY Country
ORDER BY AverageScore DESC;
```

MIN / MAX - Minimum and Maximum Values

Purpose: Find smallest or largest values

Syntax:

```
SELECT
    MIN(Sales) AS MinimumSales,
    MAX(Sales) AS MaximumSales
FROM Sales.Orders;
```

Complete Aggregate Example

```
-- Comprehensive aggregation
SELECT
    CustomerID,
    COUNT(*) AS TotalOrders,
    SUM(Sales) AS Total_Sales,
    AVG(Sales) AS AVG_Order_Value,
    MAX(Sales) AS Highest_Order,
    MIN(Sales) AS Lowest_Order
FROM Sales.Orders
GROUP BY CustomerID;
```

Window Functions - Advanced Analytics

Definition

Window Functions: Perform calculations on a "window" (subset) of rows while maintaining row-level detail

Key Difference from GROUP BY:

- **GROUP BY**: Combines rows, loses detail
- **Window Functions**: Adds calculations to EACH row without combining

Comparison: GROUP BY vs Window Functions

```
Data:
Order_ID | Product | Sales
1        | CAPS    | 10
2        | CAPS    | 30
3        | GLOVES  | 5
4        | GLOVES  | 20

GROUP BY Result (loses detail):
Product | Total_Sales
CAPS    | 40
GLOVES  | 25

Window Function Result (keeps detail):
```

```
Order_ID | Product | Sales | Total_Sales
1        | CAPS    | 10    | 40
2        | CAPS    | 30    | 40
3        | GLOVES  | 5     | 25
4        | GLOVES  | 20    | 25
```

Window Function Syntax

Basic Syntax:

```sql
SELECT
    column1,
    column2,
    WINDOW_FUNCTION(expression) OVER(
        [PARTITION BY partition_column]
        [ORDER BY order_column]
        [ROWS/RANGE BETWEEN frame_start AND frame_end]
    ) AS window_result
FROM table_name;
```

Components:

1. **WINDOW_FUNCTION**: Aggregate, Ranking, or Value function
2. **PARTITION BY**: Divides data into groups (like GROUP BY)
3. **ORDER BY**: Sort within partition
4. **FRAME CLAUSE**: Define specific rows to include (ROWS/RANGE)

Categories of Window Functions

Category	Functions	Purpose
Aggregate	SUM, AVG, COUNT, MIN, MAX	Summarize within window
Ranking	ROW_NUMBER, RANK, DENSE_RANK, NTILE, CUME_DIST, PERCENT_RANK	Rank/distribute rows
Value	LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE	Access specific rows

---

1. PARTITION BY - Dividing into Windows

Purpose: Divide result set into groups, applying calculations independently to each group

Syntax:

```sql
SELECT
    OrderID,
    ProductID,
    SUM(Sales) OVER(PARTITION BY ProductID) AS Total_Sales_by_Product
FROM Sales.Orders;
```

Examples:

```sql
-- Total sales across entire dataset (no partition)
SELECT
    OrderID,
    SUM(Sales) OVER() AS Total_Sales
FROM Sales.Orders;

-- Total sales for each product
SELECT
    OrderID,
    ProductID,
    Sales,
    SUM(Sales) OVER(PARTITION BY ProductID) AS Product_Total
FROM Sales.Orders;

-- Total sales by product AND order status
SELECT
    OrderID,
    ProductID,
    OrderStatus,
    SUM(Sales) OVER(PARTITION BY ProductID, OrderStatus) AS Product_Status_Total
FROM Sales.Orders;
```

---

2. ORDER BY - Sorting Within Windows

Purpose: Sort rows within each partition for ranking and value functions

Example:

```sql
-- Rank orders by sales (highest to lowest)
SELECT
    OrderID,
    Sales,
    RANK() OVER(ORDER BY Sales DESC) AS Sales_Rank
FROM Sales.Orders;
```

---

3. FRAME CLAUSE - Defining Row Scope

Purpose: Specify which rows within a window to include in calculations (running totals, moving averages)

Syntax:

```
SELECT
    Value,
    SUM(Value) OVER(
        ORDER BY Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS Running_Total
FROM Sales_Data;
```

Frame Types:

- **ROWS**: Physical rows
- **RANGE**: Logical range based on values

Frame Boundaries:

- `UNBOUNDED PRECEDING`: All rows from start to current
- `N PRECEDING`: N rows before current
- `CURRENT ROW`: Only current row
- `N FOLLOWING`: N rows after current
- `UNBOUNDED FOLLOWING`: All rows to end

Example - Running Total:

```
/* Data:
   Month  | Sales
   Jan    | 20
   Feb    | 10
   Mar    | 30
   Apr    | 5

   Calculation:
   Jan: 20 (just Jan)
   Feb: 20+10 = 30 (Jan+Feb)
   Mar: 20+10+30 = 60 (Jan+Feb+Mar)
   Apr: 20+10+30+5 = 65 (All) */

SELECT
    Month,
    Sales,
    SUM(Sales) OVER(
        ORDER BY Month
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS Running_Total
FROM Sales_Data;
```

4. Ranking Functions

ROW_NUMBER - Unique Sequential Numbers

Purpose: Assign unique sequential number to each row within partition

Syntax:

```
SELECT
    OrderID,
    Sales,
    ROW_NUMBER() OVER(ORDER BY Sales DESC) AS Row_Num
FROM Sales.Orders;
```

Use Cases:

Top N Analysis:

```
-- Get top 3 products by sales
SELECT *
FROM (
    SELECT
        ProductID,
        SUM(Sales) AS Total_Sales,
        ROW_NUMBER() OVER(ORDER BY SUM(Sales) DESC) AS Rank
    FROM Sales.Orders
    GROUP BY ProductID
) t
WHERE Rank <= 3;
```

Bottom N Analysis:

```
-- Get bottom 2 customers by sales
SELECT *
FROM (
    SELECT
        CustomerID,
        SUM(Sales) AS Total_Sales,
        ROW_NUMBER() OVER(ORDER BY SUM(Sales) ASC) AS Rank
    FROM Sales.Orders
    GROUP BY CustomerID
) t
WHERE Rank <= 2;
```

Duplicate Detection:

```sql
-- Identify duplicate orders
SELECT *
FROM (
    SELECT
        OrderID,
        ProductID,
        OrderDate,
        ROW_NUMBER() OVER(PARTITION BY OrderID ORDER BY CreationTime) AS rn
    FROM Sales.OrdersArchive
) t
WHERE rn > 1;  /* Show only duplicates */
```

Generate Unique IDs:

```sql
-- Assign unique pagination IDs
SELECT
    OrderID,
    ROW_NUMBER() OVER(ORDER BY OrderID, OrderDate) AS Unique_ID
FROM Sales.OrdersArchive;
```

NTILE - Distribution into Buckets

Purpose: Divide rows into N approximately equal groups (buckets)

Syntax:

```sql
SELECT
    OrderID,
    Sales,
    NTILE(N) OVER(ORDER BY Sales DESC) AS Bucket
FROM Sales.Orders;
```

Bucket Calculation:

- Bucket size = Total rows / Number of buckets
- Larger groups come first if uneven distribution

Use Cases:

Data Segmentation:

```sql
-- Segment orders into High/Medium/Low sales
SELECT *
FROM (
    SELECT
```

```
        OrderID,
        Sales,
        NTILE(3) OVER(ORDER BY Sales DESC) AS Sales_Rank
    FROM Sales.Orders
) t
-- Replace numbers with labels
SELECT
    OrderID,
    Sales,
    CASE Sales_Rank
        WHEN 1 THEN 'HIGH'
        WHEN 2 THEN 'MEDIUM'
        WHEN 3 THEN 'LOW'
    END AS Sales_Category
FROM (...);
```

Load Balancing:

```
-- Divide large table export into 2 groups
SELECT *,
    NTILE(2) OVER(ORDER BY OrderID) AS Export_Group
FROM Sales.Orders;
/* Group 1 = first half, Group 2 = second half */
```

CUME_DIST / PERCENT_RANK - Percentage-Based Ranking

Purpose: Calculate distribution percentage within window

CUME_DIST (Cumulative Distribution):

```
SELECT
    OrderID,
    Sales,
    CUME_DIST() OVER(ORDER BY Sales DESC) AS Distribution
FROM Sales.Orders;
```

PERCENT_RANK:

```
-- Find products in top 40% by price
SELECT
    ProductID,
    Product,
    Price,
    PERCENT_RANK() OVER(ORDER BY Price DESC) AS Price_Rank
FROM Sales.Products
WHERE PERCENT_RANK() OVER(ORDER BY Price DESC) <= 0.4;
```

---

5. Value Functions - Accessing Specific Rows

LAG - Access Previous Row

Purpose: Get value from previous row for comparison

Syntax:

```
SELECT
    OrderID,
    OrderDate,
    LAG(OrderDate) OVER(ORDER BY OrderDate) AS Previous_Order_Date,
    DATEDIFF(DAY, LAG(OrderDate) OVER(ORDER BY OrderDate), OrderDate) AS
Days_Between
FROM Sales.Orders;
```

Use Case - Time Gap Analysis:

```
-- Find days between consecutive orders
SELECT
    OrderID,
    OrderDate AS Current_Order,
    LAG(OrderDate) OVER(ORDER BY OrderDate) AS Previous_Order,
    DATEDIFF(DAY, LAG(OrderDate) OVER(ORDER BY OrderDate), OrderDate) AS Gap_Days
FROM Sales.Orders;
```

LEAD - Access Next Row

Purpose: Get value from next row

Syntax:

```
SELECT
    OrderID,
    OrderDate,
    LEAD(OrderDate) OVER(ORDER BY OrderDate) AS Next_Order_Date
FROM Sales.Orders;
```

FIRST_VALUE / LAST_VALUE - First and Last in Window

Purpose: Get first or last value within window

Example:

```
SELECT
    OrderID,
    ProductID,
    Sales,
    FIRST_VALUE(Sales) OVER(PARTITION BY ProductID ORDER BY OrderDate) AS
First_Sale,
    LAST_VALUE(Sales) OVER(PARTITION BY ProductID ORDER BY OrderDate ROWS BETWEEN
UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS Last_Sale
FROM Sales.Orders;
```

Subqueries - Nested Queries

Definition

Subquery (Inner Query): A query nested inside another query, used to prepare data for main query

Subquery Result Types

1. Scalar Subquery - Single Value

```
SELECT SUM(Sales) AS Total FROM Sales.Orders;
/* Result: One single value like 50000 */
```

2. Row Subquery - Multiple Columns, Single Row

```
SELECT * FROM Sales.Orders WHERE OrderID = 5;
/* Result: One row with multiple columns */
```

3. Table Subquery - Multiple Rows and Columns

```
SELECT * FROM Sales.Orders;
/* Result: Full table with many rows and columns */
```

Subquery Locations and Uses

1. Subquery in FROM Clause - Creating Temporary Table

Purpose: Prepare filtered/aggregated data before main query

Syntax:

```
SELECT col1, col2
FROM (
    SELECT col1, col2 FROM table1 WHERE condition
) AS alias;  /* Requires alias for derived table */
```

Example:

```
-- Find products more expensive than average
SELECT
    ProductID,
    Product,
    Price,
    AvgPrice,
    Price - AvgPrice AS Price_Difference
FROM (
    SELECT
        ProductID,
        Product,
        Price,
        AVG(Price) OVER() AS AvgPrice
    FROM Sales.Products
) AS t
WHERE Price > AvgPrice;

-- Rank customers by total sales
SELECT *, RANK() OVER(ORDER BY TotalSales DESC) AS Rank
FROM (
    SELECT
        CustomerID,
        SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
) AS t;
```

---

2. Subquery in SELECT Clause - Side-by-Side Comparison

Purpose: Add aggregated column alongside main query data

Rule: Only SCALAR subqueries allowed (must return single value)

Syntax:

```
SELECT
    col1,
    col2,
```

```
    (SELECT aggregate_function(col) FROM table2) AS aggregated_value
FROM table1;
```

Example:

```
-- Show total orders count next to each product
SELECT
    ProductID,
    Product,
    Price,
    (SELECT COUNT(OrderID) FROM Sales.Orders) AS TotalOrders
FROM Sales.Products;
```

---

3. Subquery in JOIN Clause - Preparing Data for Join

Purpose: Filter/aggregate data before joining with another table

Example:

```
-- Show customer details with order count
SELECT
    c.*,
    o.TotalOrders
FROM Sales.Customers AS c
LEFT JOIN (
    SELECT
        CustomerID,
        COUNT(OrderID) AS TotalOrders
    FROM Sales.Orders
    GROUP BY CustomerID
) AS o
ON c.CustomerID = o.CustomerID;
```

---

4. Subquery in WHERE Clause - Complex Filtering

Purpose: Filter rows based on aggregated or complex conditions

Rule: Only SCALAR subqueries allowed

Comparison Operators:

```
-- Find products more expensive than average
SELECT *
FROM Sales.Products
WHERE Price > (SELECT AVG(Price) FROM Sales.Products);
```

```sql
-- Find orders with below-average sales
SELECT *
FROM Sales.Orders
WHERE Sales < (SELECT AVG(Sales) FROM Sales.Orders);
```

IN Operator - Check Against List:

```sql
-- Find orders made by customers in Germany
SELECT *
FROM Sales.Orders
WHERE CustomerID IN (
    SELECT CustomerID FROM Sales.Customers WHERE Country = 'Germany'
);
```

ANY / ALL Operators:

```sql
-- Find female employees earning more than ANY male employee
SELECT *
FROM Sales.Employees
WHERE Gender = 'F'
AND Salary > ANY(
    SELECT Salary FROM Sales.Employees WHERE Gender = 'M'
);

-- Find female employees earning more than ALL male employees
SELECT *
FROM Sales.Employees
WHERE Gender = 'F'
AND Salary > ALL(
    SELECT Salary FROM Sales.Employees WHERE Gender = 'M'
);
```

EXISTS Operator - Check Row Existence:

```sql
-- Find orders made by customers in Germany
SELECT *
FROM Sales.Orders AS o
WHERE EXISTS (
    SELECT 1 FROM Sales.Customers AS c
    WHERE Country = 'Germany'
    AND o.CustomerID = c.CustomerID
);
```

```sql
-- Find orders NOT made by customers in Germany
SELECT *
FROM Sales.Orders AS o
WHERE NOT EXISTS (
    SELECT 1 FROM Sales.Customers AS c
    WHERE Country = 'Germany'
    AND o.CustomerID = c.CustomerID
);
```

Correlated vs Non-Correlated Subqueries

Non-Correlated Subquery

Characteristic: Runs ONCE independently, doesn't reference outer query

Performance: Better (executes once)

Example:

```sql
-- Average price calculated once
SELECT *
FROM Sales.Products
WHERE Price > (SELECT AVG(Price) FROM Sales.Products);
/* Subquery runs ONCE, gets average, then filters products */
```

Correlated Subquery

Characteristic: Runs for EACH row, references outer query columns

Performance: Slower (executes multiple times)

Example:

```sql
-- Check orders per customer (runs once per customer)
SELECT *,
    (SELECT COUNT(OrderID) FROM Sales.Orders AS o
     WHERE c.CustomerID = o.CustomerID) AS TotalOrders
FROM Sales.Customers AS c;
/* Subquery runs ONCE FOR EACH customer row */
```

Subquery Summary

Use Subqueries for:

1. Create temporary result sets (FROM clause)

2. Side-by-side data comparison (SELECT clause)
3. Prepare data before joining (JOIN clause)
4. Complex filtering (WHERE clause)
5. Checking row existence (EXISTS)

Avoid Subqueries When:

- Simple JOINs are sufficient
- Window functions can accomplish same goal
- CTE (Common Table Expression) is cleaner

End of Comprehensive SQL Notes

Total Topics Covered: 14 major categories **Subtopics**: 50+ detailed concepts **Examples**: 200+ real-world SQL examples **Definitions**: Complete explanations for all functions and operators

This document represents your complete SQL learning journey covering foundational to advanced concepts in SQL Server!